maktaba-web-local (المكتبة الناطقة) — Complete Algorithm Catalog

> A full sweep of the repository: retrieval/relevance core, query understanding, embeddings & inference, language generation & dialect, ingestion, study tools, security, and optional web/voice. For each algorithm: **Name** · **Location** · **Type** · **What it does**. Built from a deep, line-level audit of the actual implementations (June 2026).

Honesty legend (novelty, audited — no inflation): ★ genuine novelty (defensible for a paper/patent) · **~ incremental** (a sound but known idea, lightly extended) · **= standard** (textbook / off-the-shelf technique, used as-is) · **[!] dead/unused** (present in source but not compiled, exported, or called on the shipping build).

---

0) Architectural Context

A fully-offline, on-device, **multi-tenant** Arabic+English digital-library RAG system (FastAPI backend, Vanilla-JS SPA). Request flow: `Browser` → `middleware` → `/chat/stream (SSE)` → `guards` → `context (RAG)` → `HybridRetriever` → `offline LLM` → `dialect post-processor` → `token stream`. Every stage filters by `user_id` (strict per-tenant isolation). Runs on CPU with Ollama/llama.cpp locally, optional Groq/OpenAI/Anthropic.

---

1) Retrieval & Relevance Algorithms (the core — where the real IP is)

Algorithm	Location	Type	What it does
★ Self-calibrating per-tenant relevance gate	`relevance_gate.py`	label-free threshold calibration (the flagship invention)	Derives the accept/reject cutoff τ per tenant from the relevance scorer's own score geometry: an **in-domain self-match anchor** (a chunk's first 12 words scored against itself = guaranteed-relevant upper band) vs an **out-domain cross-book anchor** (lower band); if the bands separate, places τ a fraction α=0.25 into the gap, else a safe default. No labels, no feedback, cold-start, on-device. Cached per `(tenant, scorer, corpus-state)`. **Measured: F1 0.968 vs 0.828 fixed default; precision 0.71→0.94.**

Algorithm	Location	Type	What it does
★ Scorer-agnostic calibration (`ScorerProfile`)	`relevance_gate.py`	abstraction over the relevance model	The *same* calibration runs on an unbounded cross-encoder logit (clamp [−8,2]) **or** a bounded bi-encoder cosine (clamp [0,0.95]) reusing the loaded embedding model — no PyTorch needed → deployable on low-end devices.
★ Dual-scorer calibration disagreement (under-exploited)	`ScorerProfile`	label-free reliability / uncertainty signal	The divergence between the bi-encoder-calibrated and cross-encoder-calibrated cutoffs on the *same* tenant (e.g., the mixed-AR+EN tenant rank-flips between score spaces) is a measurable, label-free estimate of per-corpus scorer reliability. *Implemented but not yet developed as a method — strongest remaining AI angle.*
~ Per-tenant BM25 sub-index (starvation fix)	`hybrid_retriever.py:512`	sparse-retrieval correctness fix	When one tenant owns ~98% of the corpus, a shared BM25 index commits its global top-N *before* the tenant post-filter, starving the minority tenant. Builds a lazy per-tenant sparse sub-index (RAM-cached, `(id,len)`-invalidated, bounded 200 tenants) so ranking + IDF are computed within the tenant's own corpus. **Measured loss decomposition: crowding 0.39 + shared-IDF 0.14.**
~ Conditional cross-lingual fallback	`hybrid_retriever.py:66`	gated query translation	Only on a calibrated same-language miss is the query translated once (cached, 8 s timeout) and retrieval retried — translation cost is never paid on an answerable query.
~ Lexical-grounding gate + coverage routing	`context.py:327`	precision backstop + book/web fusion	Rejects a reranker "hit" with **zero** query-term overlap (a fuzzy-but-wrong match); if book coverage < 0.25 (or a recency-intent regex fires) it supplements the answer with a labelled web result (`⟦WEB⟧`).
= Hybrid retrieval (BM25 + dense + RRF)	`hybrid_retriever.py`	retrieve-then-fuse	bm25s lexical + FAISS-HNSW dense, fused by Reciprocal Rank Fusion (k=60), MD5 content dedup, over-fetch k×6 so small books reach the reranker.
= Cross-encoder reranking	`reranker.py`	neural reranker	`mmarco-mMiniLMv2-L12-H384-v1` cross-encoder scores [query,passage] pairs; drops below the gate cutoff. Library call; its value here is exposing the raw score distribution to feed the gate.

2) Query Understanding & Expansion

Algorithm	Location	Type	What it does
~ Self-extending bilingual glossary	`glossary.py`	deterministic cross-script query expansion + continual learning	308 AR→EN + 280 EN→AR domain term pairs; `cross_lingual_terms()` appends other-language equivalents to the BM25/dense query (no LLM, additive, multi-word-first). `learn_term()` persists a new pair to `data/learned_glossary.json` (bounded 5000) whenever a single-term miss triggers an LLM translation → the retrieval layer self-improves offline with no weight update.
= Arabic normalization	`hybrid_retriever.py:118`	text normalization	Strips tashkeel, unifies alef/ya/ta-marbuta, removes tatweel, lowercases — for script-robust BM25 tokenization.
= HyDE (Hypothetical Document Embeddings)	`hyde.py`	query expansion (Gao et al. 2023)	Generates a hypothetical answer passage and embeds it; merged into RRF. Optional, timeout-bounded. Faithful re-implementation.
= Multi-query expansion	`multi_query.py`	template query variants	Template/normalization variants of the query (no LLM by default). Standard.
= RAPTOR summary tree	`raptor.py`	hierarchical retrieval (Sarthi et al. 2024)	k-means clusters leaf chunks → LLM-summarizes → recurses; summary nodes stored as ordinary chunks, placed first in context. `_MAX_L1_CLUSTERS=6`. Faithful re-implementation.

---

3) Embedding & Inference

Algorithm	Location	Type	What it does
= ONNX-Runtime embeddings	`native_embeddings.py`	inference wrapper	`paraphrase-multilingual-MiniLM-L12-v2` via ONNX Runtime (CPU default, optional CUDA/DirectML). Mean-pool + L2-norm baked into the graph.
~ FP16 query-vector cache	`native_embeddings.py:114`	memory optimization	512-entry FIFO cache of query embeddings stored as float16 (768 B vs 1536 B); cosine error < 0.1%.

Algorithm	Location	Type	What it does
~ Crash-isolated GPU probing + VRAM-gated selection + OOM backoff	`native_embeddings.py:164`	safe MLOps plumbing	Probes CUDA/DirectML in a **subprocess** so a cuDNN ABI crash can't kill the server; CUDA only if free VRAM $\geq$ 800 MB; halves batch size on OOM; pauses background batches for live chat. Clever engineering, not ML novelty.
= INT8 dynamic quantization	`export_embedding_onnx.py:140`	model compression	`onnxruntime.quantization.quantize_dynamic(QInt8)` for the *mobile* embedding asset. Stock library call.
= Native C++ Arabic-aware chunk-boundary kernel	`document_optimizer.cpp`	UTF-8 sentence splitting	`smart_chunk` snaps chunk boundaries to Arabic (؛ ؟) / ASCII sentence enders at UTF-8 codepoint boundaries; pure-Python `RecursiveCharacterTextSplitter` fallback. The one native component that genuinely earns its keep.
[!] Native C++ transformer kernels + INT8/INT4 quantizer	`operator_kernels.cpp`	(NEON GEMM/softmax/RMSNorm/RoPE/SwiGLU, group quant)	**Dead code:** not compiled on the Linux build, not exported to Python, never called (inference is 100% ONNX Runtime; `universal_bridge.run_inference(float*)` is an explicit stub). Aspirational scaffolding for an Android/NPU port. **Do not claim as a contribution.**

---

4) Language Generation & Dialect

Algorithm	Location	Type	What it does
= Multi-provider LLM client	`offline_llm.py`	provider abstraction	Chain llama.cpp → Ollama → Groq → OpenAI → Anthropic; strips `<think>` tags / Chinese preamble; SSE streaming.
~ Model tiering + RAM-aware hot-swap	`offline_llm.py:207`	runtime model selection	light/balanced/max = qwen2.5 3b/7b/14b; recommends a tier from available RAM (~0.7 GB/B); hot-swaps without restart (unloads prior via `keep_alive:0`). Sensible MLOps config.
~ Safe Iraqi-dialect post-processor	`dialect_processor.py`	deterministic MSA→Iraqi rewriting	22 boundary-guarded `(?<![ء-ي])…([ء-ي]!?)` morphological rules + a 321-entry length-sorted substitution map + a conservative spacing fixer; code spans preserved. **Safe-rewriting design:** corrupts 0/21 & 0/18 common words where unguarded rules corrupt 21/10 (historical 17/14).

Algorithm	Location	Type	What it does
= Few-shot dialect prompt (greeting-scoped)	`dialect_processor.py:176`	prompt engineering	Injects curated Iraqi examples scoped to greetings only, so the model doesn't leak greetings into technical answers.
[!] MARBERT zero-shot dialect detector	`marbert_detector.py`	neural dialect ID	CLS-embedding cosine to reference sentences + keyword fast-path. **Disabled** — `server_backend.py:206` notes input-dialect detection was removed as unnecessary. Not in the live path.

---

5) Ingestion, Indexing & Caching

Algorithm	Location	Type	What it does
= PDF → text → chunks → vector store	`ingestion.py`	ingestion pipeline	PyMuPDF text extraction → smart_chunk → embeddings → Qdrant; background thread; progress persisted to `ingestion_ledger.json`.
~ Incremental FAISS build + BM25 cache + staleness check	`hybrid_retriever.py:355`	index lifecycle	BM25 corpus cached to `bm25_cache/corpus.json`; FAISS-HNSW disk cache; staleness detected by comparing Qdrant count vs cached count; incremental re-tokenize on new docs (~5 s) vs full rebuild (~30 s).
= Embedded single-process Qdrant	`vector_db.py`	offline vector store	`QdrantClient(path=…)` runs the vector engine inside the FastAPI process (no server, no network); `_clear_stale_lock()` clears a stale lock from a hard kill; optional server mode via `QDRANT_URL`.

---

6) Study-Tool / Pedagogy Algorithms

Algorithm	Location	Type	What it does
= MCQ quiz generation	`quiz.py`	LLM + structured-output validation	JSON-schema-constrained generation; truncated-array salvage parser; near-duplicate-option rejection (word overlap); meta-question rejection. **Deliberately bypasses the relevance gate** to cover the whole book.
= SM-2 spaced repetition	`srs.py`	scheduling (Woźniak 1990)	Verbatim SuperMemo-2 interval formula for flashcard review. Standard, correctly cited.

7) Security & Guard Algorithms

Algorithm	Location	Type	What it does
~ Auth: bcrypt + constant-time dummy hash	`auth.py`	timing-safe authentication	bcrypt (cost 12); runs a dummy bcrypt even for unknown users to defeat timing-based account enumeration.
= SHA-256 hashed session tokens + HttpOnly cookies	`auth.py`	session security	32-byte random tokens stored SHA-256-hashed at rest (the DB never holds the raw token; the raw value is carried in an HttpOnly cookie for web, or a Bearer header for mobile); legacy plaintext rows upgraded in place; no JWT.
= ASGI middleware stack	`middleware.py`	web security	CSP/security headers, CSRF origin-check, request logging, per-IP rate limit (skips `/static/`).
= Circuit breaker / dedup / rate limiter	`guards.py`	reliability guards (stdlib only)	Ollama circuit breaker (opens after 5 failures, resets after 30 s); duplicate-request dedup (session+message hash); per-user chat rate limit; per-IP registration limit.

8) Helper & Statistical Algorithms

Algorithm	Location	Type	What it does
= Reciprocal Rank Fusion	`hybrid_retriever.py:978`	rank fusion (Cormack 2009)	Fuses BM25 + dense (+ HyDE + variants) rankings by `Σ 1/(k+rank)`, k=60.
= Cosine similarity	`hybrid_retriever.py:445`	similarity	Bi-encoder relevance scoring for the gate's lightweight mode.
= k-means clustering	`raptor.py`	clustering (sklearn)	Groups leaf chunks for RAPTOR summarization.
= Bootstrap CI / paired significance	"	statistics	2000-sample bootstrap 95% CIs and per-query paired tests for the papers' evaluations.

9) Optional Web / Voice Algorithms

Algorithm	Location	Type	What it does
= Duck-DuckGo web fallback	`context.py`	web search	When a query isn't grounded in the books, searches the web and labels the source explicitly.
= Piper neural TTS / faster-whisper STT	`'| offline speech | Neural text-to-speech (AR+EN) and offline transcription; degrade to 503 if absent. | | ** = XTTS voice (run_xtts.sh)** |run_xtts.sh'`	neural TTS	Coqui XTTS voice — fits the name "the speaking library." *Never analyzed as a research/novelty candidate; an unexplored offline-Arabic-dialect-TTS angle that would need fresh evaluation before any claim.*

Numeric Summary & Honest Novelty Ledger

- **Total catalogued mechanisms:** ~40 across 9 categories.
- **★ Genuine novelty (paper/patent-worthy):** the **self-calibrating per-tenant relevance gate**, its **scorer-agnostic** form, and the **dual-scorer calibration-disagreement** signal — all in the retrieval/relevance core. *This is the project's real IP.*
- **~ Incremental (defensible as focused systems/IR/NLP contributions):** per-tenant BM25 sub-index (starvation), self-extending glossary (continual cross-lingual learning), safe Iraqi-dialect rewriting, conditional cross-lingual fallback, lexical-grounding/coverage routing, FP16 cache + crash-isolated GPU probing, model tiering.
- **= Standard / off-the-shelf:** RRF, HyDE, RAPTOR, multi-query, cross-encoder reranking, ONNX/INT8, SM-2, MCQ validation, the full security stack, embedded Qdrant.
- **[!] Dead / not-shipping (must NOT be claimed):** the native C++ transformer kernels + INT8/INT4 quantizer (uncompiled on Linux, unexported, uncalled); the MARBERT dialect detector (disabled in `server_backend.py`).

Audit caution: do not market quantization, LoRA, distillation (absent entirely), SM-2, the native kernels, or the MARBERT detector as novelty — they will not survive review or a patent examiner. The defensible contributions are the relevance-calibration family (★) and the three focused systems/IR/NLP mechanisms (~) already written up as papers in `~/Desktop/14-6/`.